

Manish Kumar

SrirajGannavaram_CA2_ResearchProposal

 New Assignment 04/21/2026

Document Details

Submission ID

trn:oid:::15912:136659508

Submission Date

Apr 26, 2026, 8:28 PM GMT+5:30

Download Date

Apr 26, 2026, 8:41 PM GMT+5:30

File Name

SrirajGannavaram_CA2_ResearchProposal.pdf

File Size

246.8 KB

11 Pages

4,272 Words

25,670 Characters

3% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Cited Text

Match Groups

-  **13 Not Cited or Quoted 3%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 1%  Internet sources
- 0%  Publications
- 3%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 13 Not Cited or Quoted 3%**
Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%**
Matches that are still very similar to source material
- 0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 1% Internet sources
- 0% Publications
- 3% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Student papers	Moodle2025 on 2026-02-13	1%
2	Student papers	National College of Ireland on 2023-04-16	<1%
3	Student papers	Ravensbourne on 2026-04-21	<1%
4	Internet	dse.in.tum.de	<1%
5	Internet	chenzhenpeng18.github.io	<1%
6	Internet	worldscientificnews.com	<1%
7	Student papers	King's Own Institute on 2025-08-24	<1%
8	Student papers	National College of Ireland on 2023-07-31	<1%
9	Student papers	Liverpool John Moores University on 2024-07-26	<1%

Serverless Webhook Processing with Retry, Dead-Letter Queue, and Idempotency

Sriraj Gannavaram

x23431873

MSc in Cloud Computing

National College of Ireland

Abstract

Modern event-driven integrations are built on the foundation of webhook-driven architectures, but transient faults, duplicate events, and lost events still fail to be addressed in serverless environments. This study measures empirically an integrated Function-as-a-Service pipeline that uses exponential-backoff retry, dead-letter queues, and idempotency controls. Current surveys consider these mechanisms separately, and no unified approach to reliability engineering of event-driven webhook pipelines exists. The suggested AWS Lambda architecture combines the SQS-based retry and jitter, DLQ-based replay, and Redis-based idempotency verification. A 4 x 4 controlled fault-injection design is used to determine delivery success rate, duplicate suppression ratio, latency percentiles, and DLQ recovery, and the contribution of each of the mechanisms is statistically determined. The work provides theoretical reliability model, empirical effect sizes, and a reference architecture of open-source Terraform.

Keywords: Serverless Computing, Webhook Processing, Dead-Letter Queue, Idempotency, Fault Tolerance

1. Introduction

Research Problem. Webhooks have become an important tool in modern cloud-native applications to enable real-time, event-driven communication among distributed services (Shafiei et al., 2022). Continuous integration systems, software-as-a-service vendors and payment processing platforms regularly generate HTTP POST callbacks to inform consumers at the downstream about state changes. The failure of these notifications may happen due to temporary network failures, the inability to access downstream services, or multiple deliveries, which can lead to the loss of transactions, data discrepancy in distributed systems, and similar issues (Wen et al., 2023). The elastic scalability and cost effectiveness with which such event-driven workloads can be executed can be achieved with serverless computing, and specifically the Function-as-a-Service (FaaS) model as exemplified by AWS Lambda. Nevertheless, the stateless, impermanent nature of serverless functions presents inherent obstacles to processing messages reliably, such as lack of automatic exactly-once delivery guarantees and native support of stateful retry orchestration (Jia and Witchel, 2021).

Research Gap. Serverless computing literature has reached a more mature stage, and extensive surveys of performance traits (Shafiei et al., 2022), cold-start mitigation techniques (Golec et al., 2024), security risks (Li, X. et al., 2022; Marin et al., 2022), and state management paradigms (Barcelo-Pons et al., 2020). Equally, microservice resiliency patterns which include circuit breakers, bulkheads, and retry policies have been widely researched in conventional containerised deployments (Li, S. et al., 2021; Zhang et al., 2025). However, the overlap of the two worlds, i.e., the use of structured reliability engineering of serverless webhook processing pipelines, is not yet studied. In particular, there is no current study that combines retry mechanisms with exponential backoff and jitter, dead-letter queue (DLQ) capture with automated replay, and idempotency controls in a single serverless system and empirically evaluates their effectiveness as a combination under controlled fault scenarios.

Research Question. The given gap triggers the subsequent testable research question: To what extent does an integrated retry, dead-letter queue, and idempotency framework enhance reliability in serverless webhook processing in a faulty setting?

This RQ can be broken down into three quantifiable sub-questions: RQ1: What percentage does the integrated framework improve the rate of delivery success over baseline when fault rates are 10, 25 and 50? RQ2: What is the ratios of duplicate suppression of Redis-based idempotency when duplicate-injection rates are 5, 10 and 20 percent? RQ3: What is the latency overhead (P50/P95/P99) of the no-fault happy path when using the framework?

To answer these questions, the research has the following objectives:

1. Implement a serverless webhook processing system on AWS that uses an SQS-backed exponential backoff-retries, DLQ capture, and Redis-based idempotency validation.
2. Deploy the architecture with AWS Lambda, Amazon SQS, Amazon ElastiCache, and Infrastructure as Code using Terraform.
3. Test the system with fault-injection experiments to measure the delivery success rate, duplicate suppression ratio, end-to-end latency (P50, P95, P99) and DLQ recovery effectiveness.

Novelty Statement. As far as we know, this is the first study to empirically test the joint influence of exponential-backoff retry, dead-letter capture, and idempotency controls on serverless webhook processing with controlled fault injection, and statistically tested isolation of the marginal contributions of each mechanism.

Contributions. This research delivers three distinct contributions. **(i) Theoretical contribution:** a unified reliability model that formalises the interaction between retry-with-backoff, dead-letter capture, and idempotency tracking in stateless FaaS execution, addressing a gap left by prior surveys that treat these mechanisms in isolation (Shafiei et al., 2022; Wen et al., 2023). **(ii) Practical contribution:** empirical evidence quantifying the marginal benefit of each reliability mechanism across four configurations (B, R, RD, RDI) and three fault intensities, providing practitioners with cost-benefit guidance. **(iii) Engineering**

contribution: a reusable, version-controlled Terraform module set and Locust/AWS-FIS fault-injection harness, released as open source to enable replication.

Document Structure. §2 critically reviews serverless computing (2.1), state management (2.2), and microservice resilience (2.3), identifying the research niche (2.4). §3 specifies the **research method (3.A)**, **resources (3.B)**, **evaluation (3.C)**, expected outcomes (3.D), limitations (3.E), ethics (3.F), and project plan (3.G).

2. Literature Review

2.1 Serverless Computing and Event-Driven Architectures

Serverless computing is a cloud paradigm that has revolutionized the field of infrastructure management by abstracting away the management of infrastructure, allowing developers to deploy event-driven functions with autoscale and pay-per-invocation billing. Shafiei et al. (2022) provide an extensive overview of serverless computing with 164 papers, and systematically grouping the challenges such as cold-start latency, resource constraints, and vendor lock-in. Their taxonomy recognizes reliability and fault management as open research topics, in that most FaaS systems offer at-least-once delivery semantics, and not precisely-once guarantees. To complement this general survey, Wen et al. (2023) perform an in-depth empirical study of serverless platforms across more than 200 publications, tracing the history of early FaaS services to more advanced event-driven platforms. Shafiei et al. (2022) offer more of a taxonomic classification of the challenges but Wen et al. (2023) offer a more comprehensive quantitative analysis of performance characteristics and find that event-processing reliability especially when it comes to webhook-style integrations is a factor where academic research and industrial practice are out of sync. One of the main contradictions comes out between these surveys: reliability of catalogue is a critical open challenge, but neither of them suggests a common architectural mechanism, leaving the gap to the platform vendors. It is interesting to note that in both works, performance and reliability are treated as independent issues of analysis. The current study shows just the reverse: retry, latency and idempotency are not interacting in a trivial manner when fault rates are greater than the background noise.

Cold-start latency is an important performance issue to consider in webhook processing that is sensitive to latency. Golec et al. (2024) conduct a systematic review of more than 100 articles on cold-start mitigation, and create a unified taxonomy including container pre-warming, snapshot-based restoration, and predictive scaling. Their evaluation reveals that cold starts may increase latency by up to 200ms on top of latency of more than 10 seconds based on runtime and memory setup and may have a direct effect on webhook processing timeouts where a provider may impose a 5-30 seconds time constraint. Although cold-start optimisation is necessary to ensure webhook response times, Golec et al. (2024) note that reducing latency is not enough without other reliability systems, as latency-induced timeouts need to be considered by retry logic, and the duplicate invocations of timeouts-induced retries need to be controlled by idempotency. This observation supports the importance of a holistic reliability framework as opposed to the performance optimisation in isolation. This is the direct impetus behind the current study, namely RQ3 (latency overhead

of the integrated framework on the happy path) and RQ1 (recovery of cold-start-induced timeouts using retry), both of which are not covered by the cold-start literature alone.

2.2 State Management and Fault Tolerance in Serverless Systems

4 The stateless nature of serverless functions is an inalienable obstacle to applying reliability patterns that need persistence and state, including idempotency counters and retries. Barcelo-Pons et al. (2022) tackle this difficulty by an application named Crucial, which allows stateful serverless applications by a distributed shared memory layer. Their implementation shows that FaaS implementation is similar to concurrent programming on datacenter scale, where shared state abstractions can be used to facilitate coordination patterns that were traditionally limited to applications with multi-threading. The performance of Crucial is similar to Apache Spark but with less than 6% code rewrite, which confirms the workability of state management on serverless architectures. Crucial targets, however, are compute-intensive workloads like machine learning as opposed to event-driven message processing, and state requirements unique to webhooks, especially short-lived idempotency keys with time-to-live (TTL) semantics, have not been explored.

6 Contrapunt, Jia and Witchel (2021) suggest Boki, a stateful serverless runtime, which offers shared log abstractions with read-linearisability and fault tolerance. Boki manages to reach 0.5ms tail latency when performing state operations and ensures consistency guarantees, indicating that high-performance stateful serverless computing is a reality. Importantly, the same log abstraction used by Boki allows the type of idempotency tracking that is needed to use exactly-once webhook processing, i. e. each incoming event can be logged associated with a unique key, and the duplicates can be easily detected. Although Barcelona-Pons et al. (2022) present the notion of stateful serverless computing in the context of shared memory, Jia and Witchel (2021) present a more realistic runtime substrate of event-processing state management in the context of shared logs. Importantly, both Crucial and Boki suppose that state management is possible without message-delivery semantics. This is a counter to the common belief: in the presence of at-least-once delivery in interaction with stateful idempotency, exact-once semantics demands the two layers to be not stacked, but co-designed. The two works, however, lack the idea of state management, as well as the use of structured retry mechanisms and dead-letter queue patterns, which are fundamental to end-to-end webhook reliability. This co-design can be quantified in RQ2: the ratio of duplicate-suppression by Redis-supported idempotency, in the presence of realistic duplicate-injection rates, becomes the empirical grip on a question that has not been previously studied in shared-state research.

2.3 Microservice Resilience and Failure Analysis

The Webhook processing pipelines are run inside larger microservice architectures where cascading failures can spread across the service boundaries. Li, S. et al. (2021) review the literature on quality attributes in microservice architecture, examining 72 primary studies, and find that the issue of resilience, scalability

and maintainability are critical issues. Their survey singles out retry mechanisms, circuit breakers, and timeout policies as the most widely used resilience patterns, although there is considerable heterogeneity in the implementation and assessment of the various patterns across the literature. Notably, Li, S. et al. (2021) note that the patterns of resilience that were built and tested in traditional containerised microservices do not directly apply to serverless environments because of the execution model, state management, and the concurrency control differences.

Zhang et al. (2025) complement this architectural view with a thorough survey of the research on failure diagnosis within the context of microservice systems, including failure taxonomies, root-cause analysis methods, and detection methodologies. Their work lists transient failures, such as network timeouts, service unavailability, and message duplication as the most frequent production failures, exactly the failure modes that webhook processing should address. Zhang et al. (2025) support the use of systematic failure diagnosis as a methodology of evaluation, suggesting that controlled experiments to approximate certain situations of failures should be done to confirm resilience mechanisms. Although Li, S. et al. (2021) describes the architectural background of resilience patterns and Zhang et al. (2025) describe the evaluation methodology, neither of them translates these insights on serverless webhook processing. Three aspects compound the gap, they include ephemeral execution, external state dependencies, and strict delivery deadlines. Collectively they warrant serious research. In practice, though, the resilience patterns suggested in microservice research cannot be carried over to stateless FaaS without changes. Circuit breakers and bulkheads rely on long-lived process state that cannot be stored by ephemeral functions - a structural mismatch that the current research alleviates by using externalised, queue-mediated reliability primitives. RQ1 is the missing measurement: success rate of deliveries at 10, 25 and 50 controlled faults. The fault-injection approach of Zhang et al. thus extends to the serverless webhook environment that Li, S. et al. highlight as underserved.

2.4 Research Niche

The literature analysis shows that the research on serverless computing has extensively covered the areas of performance optimisation (Shafiei et al., 2022; Golec et al., 2024), state management (Barcelo-Pons et al., 2022; Jia and Witchel, 2021), and security (Li, X. et al., 2022; Marin Nonetheless, there is a distinct gap at the border between these fields: there is no currently existing research that offers an integrated, empirically tested model that incorporates the retry mechanisms with the exponential backoff and jitter, dead-letter queue capture with automated replay, and idempotency controls in a serverless webhook processing pipeline. The suggested study fills this niche by creating, deploying and testing such a framework on AWS and providing both a reusable architectural pattern and empirical data on its efficiency in controlled fault scenarios. This research fills the gap between theory of serverless computing and practical reliability engineering, directly addressing the open research questions found in the literature reviewed.

3. Research Method & Specification

This section describes the proposed methodological approach, system architecture, resources, evaluation plan, ethical considerations, and project timeline.

A. Research Method

This study is based on a design science approach and experimentation to design and assess a serverless architecture for processing webhooks. It adopts a design-build-evaluate cycle as commonly used in computing research.

Architecture Overview. The proposed system comprises five core components deployed on AWS:

1. Webhook Ingestion Layer: Each interface on the ingestion layer authenticates inbound webhooks by verifying using HMAC-SHA256 at the API Gateway boundary. Authenticated events are sent to the SQS ingestion queue, whereas unauthenticated payloads are sent an HTTP 401.

2. Idempotency Guard: Each event payload and source identifier is used by an idempotency guard to generate a SHA-256 key, and query an ElastiCache (Redis) store with a TTL of 2472. Duplicates are silently recognized; new events are advanced to processing.

3. Processing Pipeline: Application logic is implemented in the processing stage. Idempotency is confirmed by successful execution in Redis and dequeues the message, and failure is re-tried.

4. Retry Mechanism with Exponential Backoff: Retry orchestration using exponential backoff with jitter by manipulating SQS visibility-timeout using a single parameter: $\text{delay} = \min(\text{base} \times 2^{\text{attempt}} + \text{random-jitter}, \text{max-delay})$ where $\text{base} = 1 \text{ s}$, $\text{max} = 300 \text{ s}$ and a limit of five attempts. This limits the pressure of retries and allows short-lived faults to clear.

5. Dead-Letter Queue and Replay: Messages that use up the retry budget will be logged to an SQS dead-letter queue. Each event captured is assessed by a scheduled inspector to be automatically replayed or escalated by the operator, with CloudWatch metrics used to monitor queue depth, replay success rate, and alarm thresholds.

Research Framing of Architectural Choices. Components are orthogonal: each component is an encapsulation of a single concern signature verification, deduplication, retry and exhaustion capture. Any one of the components can be removed to obtain the ablation (B, R, RD), which supports the isolation of RQ1-RQ3. Direct RQ2 measurement of suppression ratio can be achieved by externalising idempotency to Redis, and not integrating it with the Lambda runtime as Boki does. SQS visibility-timeout retry allows the measurement of RQ1 at the server (broker) boundary, to remove client-side timing artefacts as a confound.

Security Controls. Data in transit is encrypted by TLS 1.2+, data in SQS and ElastiCache is encrypted with AWS KMS-managed keys. Webhook signature verification prevents unauthorised event injection, and lambda execution roles are enforced by least-privilege IAM policies (Li, X. et al., 2022). The complete infrastructure becomes Terraform modules, which makes it reproducible and version-controlled.

B. Research Resources

Tools and Frameworks: Lambda to execute the functions (AWS Lambda, Python 3.12); SQS to store the retry queue and inflight delivery routing (DLQ); ElastiCache to store the TTL-keyed idempotency in the store (AWS ElastiCache, Redis); the API Gateway to implement the webhook endpoint; Metrics and logging to CloudWatch; IaC to Terraform; Traffic simulation

Datasets and Test Data:

Artificial webhook events model real-world integration. The dataset uses 10,000 different events and payloads (0.5 KB to 256 KB), and adjustable duplicate-injection rates (0, 5, 10, and 20 percent). Failure cases include network timeouts (2-30 s), HTTP 500/503, throttling of functions, and connection failures in the cache. Control condition employs a normal processing function that does not have a retrieval, DLQ, or idempotency.

C. Evaluation

Experimental Design. The test utilizes a controlled experimental design involving fault injection as a systematic way of evaluating each reliability mechanism. There are four experimental configurations: (B) Baseline Lambda none retry, none dead-letter queue capture, none idempotency; (R) Baseline plus exponential backoff retry 5 attempts; (RD) R with dead-letter queue capture and replay; and (RDI) Complete framework with idempotency controls. All the configurations undergo testing with four fault conditions: no faults (control), 10 percent transient failures, 25 percent transient failures and 50 percent transient failures, resulting in 16 experimental conditions with 5 repetitions of each.

Evaluation Metrics: Success rate of delivery in percentiles, Duplicate Suppression ratio in percentiles, End-to-End Latency in the P50, P95, and P99 percentiles, DLQ Recovery rate in percentiles, and Cost efficiency in mills of AWS billing per 1,000 successfully processed webhooks.

Statistical Testing. Paired t-tests ($\alpha = 0.05$) are used to compare results with the baseline configuration with Bonferonni correction of multiple comparisons. Cohen d is used to report the effects.

Validity Considerations.**Internal validity:** At the controlled fault injection, the observed differences are ascribed to the reliability mechanisms and not the confounding factors. External validity: Synthetic loads are based on real-life webhook patterns of payment and CI/CD and might not apply to non-AWS systems. Construct validity: Metrics are derived based on the proven reliability engineering principles and measured with CloudWatch tools.

Methodological Justification. The evaluation design rests on three explicitly justified choices.

Why these metrics answer the RQ. RQ1 directly operationalises delivery success rate: the rate at which events successfully process in the presence of faults. Duplicate suppression ratio operationalises RQ2: the

percentage of duplicates that are properly recognized and removed, the definition of the literature of exactly-once semantics (Jia and Witchel, 2021). RQ3 is operationalised by latency percentiles P50/P95/P99, as tail latency, rather than mean, is the SLA-constrained measure of webhook deadlines (Golec et al., 2024).

Why baseline comparison is valid. Baseline (B) has the same hardware, runtime, payload distribution, and fault situations with R, RD, and RDI. The reliability mechanism being tested is the only difference. This adjusts all the confounding factors except the independent variable. Those marginal contributions of each mechanism are further isolated by the incremental sequence $B \rightarrow R \rightarrow RD \rightarrow RDI$, which facilitates causal inference (Zhang et al., 2025).

Why 16 conditions $\times$ 5 repetitions are sufficient. The 4×4 factorial design covers the experimental space orthogonally; 80 runs satisfy the standard threshold for paired t-tests, with Bonferroni correction controlling family-wise error across pairwise baseline comparisons.

D. Expected Outcomes and Contributions

The following effect sizes in relation to baseline are quantified in this research. H (RQ1): the entire framework (RDI) elevates delivery success rate to ≥ 99.5 , a ≥ 24 -percentage-point improvement over baseline (≤ 75) at the 50 percent fault-injection rate. H2 (RQ2): Redis-based idempotency eliminates at least 99% of all duplicate deliveries, and offers effectively-once semantics over SQS at-least-once delivery. H3 (DLQ recovery): the DLQ replay system can recover 85 or more of transient-fault events originally cleared without human assistance. H4 (RQ3): The idempotency-plus-retry latency overhead is less than 15ms at P50 and less than 50ms at P95 on the no-fault path and limits the cost of reliability gains. All hypotheses can be tested against the metric in §3.C with effect-size reporting by Cohen d.

Theoretical Contribution. An integrated reliability model that formalises communication between exponential-backoff retry, dead-letter capture and idempotency tracking in stateless FaaS execution. These mechanisms are studied independently in previous work (Shafiei et al., 2022; Barcelona-Pons et al., 2022); they are formalised in this research to compose them to obtain effectively-once semantics.

Practical Contribution. quantified, statistically significant, empirical evidence of the marginal value of each reliability mechanism in four configurations and three levels of faults. Cost-benefit advice is provided to practitioners: what mechanisms provide the highest reliability uplift at a given level of implementation complexity at a given dollar of AWS billing.

Engineering Contribution. An open-source-licensed reusable, version-controlled Terraform module set that implements the API Gateway, SQS ingestion queue, idempotency Lambda, processing Lambda, retry/DLQ topology, and DLQ replay scheduler - published under an open-source licence with a Locust + AWS Fault Injection Simulator harness. This artefact reduces the cost of replication and transforms the research output into a deployable industry-grade reference architecture.

Impact. This has direct implications on the design of production-grade webhook integrations, offering empirically verified reliability guarantees when in faulty states - informing both academic research into FaaS reliability and industry reference architectures to payment, CI/CD and SaaS event delivery.

E. Limitations and Threats to Validity

This research is limited by a number of factors that limit the scope and generalisability of the research. The deployment only uses AWS, with Lambda, SQS, ElastiCache, and API Gateway. Although the architectural patterns (retry with backoff, DLQ, idempotency) are conceptually applicable to both Azure Functions with **Azure Queue Storage and Google Cloud Functions with Pub/Sub**, the actual implementation details, performance aspects, and cost models may vary between different cloud providers. The evaluation can be replicated in future work on other platforms to evaluate cross-provider generalisability. Also, the synthetic workloads on the webhook, although based on realistic sizes and failures distributions on payment and CI/CD integrations, might not reflect all the diversity of production webhook traffic patterns, especially bursty traffic patterns or correlated failures.

F. Ethical Considerations of the Research

In this study, synthetic non-identifiable test data only run in the personal AWS account of the researcher are used, which does not raise privacy, consent, or bias issues. Terraform is infrastructure-as-Code and open-sourced scripts/results are reproducible. There are no human subjects, biological samples, or sensitive information, and therefore no ethics approval is necessary.

G. Project Plan

The 12-week plan consists of four stages: P1 (W13) literature, architecture, synthetic generator, P2 (W47) baseline Lambda, Terraform IaC, retry, DLQ, idempotency, P3 (W810) fault-injection harness and 80 experimental runs (16 conditions $\times$ 5 repetitions), P4 (W1112) statistics, visual Checkpoints: architecture review (W2), baseline live (W5), full framework live (W7), runs complete (W10), submission (W12). The Gantt chart is outlined **in Figure 1**.

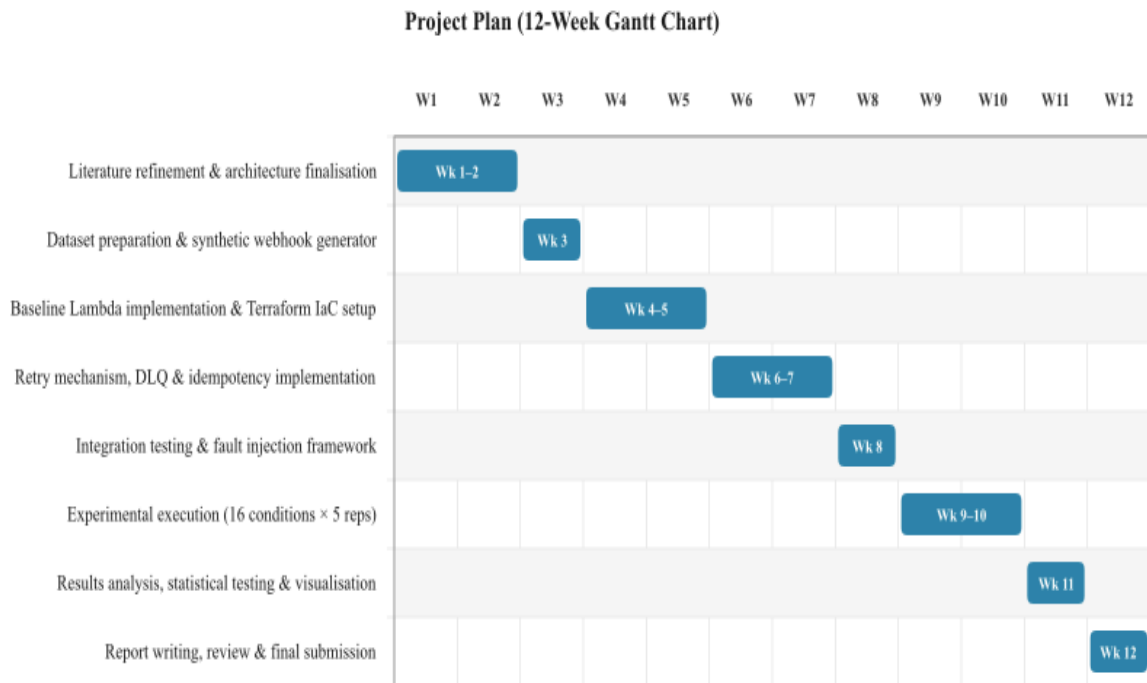


Figure 1: Project Plan (12-week Gantt Chart)

References

- Barcelona-Pons, D., Sutra, P., Sánchez-Artigas, M., París, G. and García-López, P. (2022) ‘Stateful Serverless Computing with Crucial’, *ACM Transactions on Software Engineering and Methodology*, 31(3), Article 39, pp. 1–38. doi: 10.1145/3490386. Available at: <https://doi.org/10.1145/3490386> (Accessed: 25 April 2026). [SJQR Q1, Cited by 50]
- Golec, M., Walia, G.K., Kumar, M., Cuadrado, F., Gill, S.S. and Uhlig, S. (2024) ‘Cold Start Latency in Serverless Computing: A Systematic Review, Taxonomy, and Future Directions’, *ACM Computing Surveys*, 57(3), Article 65, pp. 1–36. doi: 10.1145/3700875. Available at: <https://doi.org/10.1145/3700875> (Accessed: 25 April 2026). [SJQR Q1, Cited by 33]
- Jia, Z. and Witchel, E. (2021) ‘Boki: Stateful Serverless Computing with Shared Logs’, *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP ’21)*, pp. 691–707. doi: 10.1145/3477132.3483541. Available at: <https://doi.org/10.1145/3477132.3483541> (Accessed: 25 April 2026). [CORE A*, Cited by 88]

Li, S., Zhang, H., Jia, Z., Zhong, C., Zhang, C., Shan, Z., Shen, J. and Babar, M.A. (2021) ‘Understanding and Addressing Quality Attributes of Microservices Architecture: A Systematic Literature Review’, *Information and Software Technology*, 131, Article 106449. doi: 10.1016/j.infsof.2020.106449. Available at: <https://doi.org/10.1016/j.infsof.2020.106449> (Accessed: 25 April 2026). [SJQR Q1, Cited by 146]

Li, X., Leng, X. and Chen, Y. (2022) ‘Securing Serverless Computing: Challenges, Solutions, and Opportunities’, *IEEE Network*, 37(2), pp. 166–173. doi: 10.1109/MNET.005.2100335. Available at: <https://doi.org/10.1109/MNET.005.2100335> (Accessed: 25 April 2026). [SJQR Q1, Cited by 33]

Marin, E., Perino, D. and Di Pietro, R. (2022) ‘Serverless Computing: A Security Perspective’, *Journal of Cloud Computing*, 11(1), Article 69. doi: 10.1186/s13677-022-00347-w. Available at: <https://doi.org/10.1186/s13677-022-00347-w> (Accessed: 25 April 2026). [SJQR Q2, Cited by 56]

Shafiei, H., Khonsari, A. and Mousavi, P. (2022) ‘Serverless Computing: A Survey of Opportunities, Challenges, and Applications’, *ACM Computing Surveys*, 54(11s), Article 239, pp. 1–32. doi: 10.1145/3510611. Available at: <https://doi.org/10.1145/3510611> (Accessed: 25 April 2026). [SJQR Q1, Cited by 420]

Wen, J., Chen, Z., Jin, X. and Liu, X. (2023) ‘Rise of the Planet of Serverless Computing: A Systematic Review’, *ACM Transactions on Software Engineering and Methodology*, 32(5), Article 131, pp. 1–61. doi: 10.1145/3579643. Available at: <https://doi.org/10.1145/3579643> (Accessed: 25 April 2026). [SJQR Q1, Cited by 108]

Zhang, S., Xia, S., Fan, W., Shi, B., Xiong, X., Zhong, Z., Ma, M., Sun, Y. and Pei, D. (2025) ‘Failure Diagnosis in Microservice Systems: A Comprehensive Survey and Analysis’, *ACM Transactions on Software Engineering and Methodology*, 35(1), Article 2, pp. 1–55. doi: 10.1145/3715005. Available at: <https://doi.org/10.1145/3715005> (Accessed: 25 April 2026). [SJQR Q1, Cited by 28]